

■ Day 3 — Redshift Distribution Styles & Industry Practice

■ Topics Covered

- DISTSTYLE EVEN
 - DISTSTYLE KEY
 - DISTSTYLE ALL
 - DISTSTYLE AUTO
 - DISTKEY Internal Working
 - Hashing & Node Distribution
 - Data Skew Problem
 - Local Joins vs Network Shuffle
 - SORTKEY Optimization
 - Fintech Warehouse Design
-

■ DISTSTYLE EVEN

Rows distributed equally across nodes.

Used for:

- staging tables
- temporary ETL loads
- bronze layer tables

■ DISTSTYLE KEY

Rows distributed using HASH of DISTKEY column.

Internal logic:

$\text{node} = \text{hash}(\text{customer_id}) \% \text{total_nodes}$

Same IDs always land on same node.

■ DISTSTYLE ALL

Entire table copied to every node.

Best for small dimension tables.

■ DISTSTYLE AUTO

Redshift automatically decides best distribution style.

■ Internal DISTKEY Working

Suppose:
customer_id = 501

hash(501) = 928371827

$928371827 \% 4 = 3$

So Redshift stores rows in Node3.

All rows having customer_id=501 always go to same node.

■ Data Skew Problem

Bad Example:
DISTKEY(status)

Only two values exist:
SUCCESS / FAILED

Result:
Node1 overloaded
Node2 overloaded
Node3 idle
Node4 idle

■ Scenario — Fintech Payment Warehouse

Applications
↓
Amazon S3
↓
COPY Command
↓
Amazon Redshift
↓
Power BI / Tableau

■ customers.csv

```
customer_id,customer_name,city,signup_date
101,Rohan,Mumbai,2025-01-10
102,Amit,Pune,2025-02-15
103,Neha,Delhi,2025-03-20
104,Priya,Bangalore,2025-04-05
105,Karan,Hyderabad,2025-05-01
```

■ transactions.csv

```
transaction_id,customer_id,merchant_id,amount,payment_mode,status,transaction_time
5001,101,9001,1200.50,UPI,SUCCESS,2026-07-01 10:00:00
5002,102,9002,2500.00,CARD,SUCCESS,2026-07-01 10:05:00
5003,101,9003,800.00,WALLET,FAILED,2026-07-01 10:10:00
5004,103,9001,1500.00,UPI,SUCCESS,2026-07-01 10:20:00
5005,104,9004,7000.00,CARD,SUCCESS,2026-07-01 10:25:00
```

■ Create customers Table

```
CREATE TABLE customers (
  customer_id BIGINT,
  customer_name VARCHAR(100),
  city VARCHAR(50),
  signup_date DATE
)
DISTSTYLE KEY
DISTKEY(customer_id);
```

■ **Create transactions Table**

```
CREATE TABLE transactions (
  transaction_id BIGINT,
  customer_id BIGINT,
  merchant_id BIGINT,
  amount DECIMAL(10,2),
  payment_mode VARCHAR(20),
  status VARCHAR(20),
  transaction_time TIMESTAMP
)
DISTSTYLE KEY
DISTKEY(customer_id)
SORTKEY(transaction_time);
```

■ **COPY customers**

```
COPY customers
FROM 's3://redshift-demo-bucket/customers.csv'
IAM_ROLE 'YOUR-IAM-ROLE-ARN'
CSV
IGNOREHEADER 1;
```

■ **COPY transactions**

```
COPY transactions
FROM 's3://redshift-demo-bucket/transactions.csv'
IAM_ROLE 'YOUR-IAM-ROLE-ARN'
CSV
IGNOREHEADER 1;
```

■ **Revenue Query**

```
SELECT
  c.customer_name,
  SUM(t.amount)
FROM customers c
JOIN transactions t
ON c.customer_id=t.customer_id
GROUP BY c.customer_name;
```

■ **Payment Mode Analytics**

```
SELECT
  payment_mode,
  COUNT(*),
  SUM(amount)
FROM transactions
GROUP BY payment_mode;
```

■ **Failed Transactions**

```
SELECT *
FROM transactions
```

```
WHERE status='FAILED';
```

■ Day 3 Outcome

- Learned all Distribution Styles
- Understood internal hashing
- Learned local joins
- Understood Data Skew
- Built Fintech Warehouse
- Loaded CSVs using COPY
- Practiced analytics queries